Recitation 6 – Web APIs, REST, DTOs, and Protocol Buffers

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. Web APIs
2. REST API
3. CRUD Operations
4. Data Transfer Objects (DTOs)
5. Protocol Buffers
6. Summary
7. REST API Design Exercise

Today

- Web APIs - Overview and Types
- REST API - Deep Dive
- CRUD Operations
- Data Transfer Objects (DTOs)
- Protocol Buffers (protobuf)
- REST API Design Exercise

What is an API?

Application Programming Interface (API)

A **contract** that defines how software components communicate with each other. It specifies:

- What operations are available
- What inputs are required
- What outputs to expect

Everyday Examples:

- Restaurant menu = API (defines what you can order)
- Electrical outlet = API (defines voltage, plug shape)
- USB port = API (defines physical and data interface)

What is a Web API?

Web API Definition

A **Web API** is an API that is accessible over the network using web protocols (typically HTTP/HTTPS).

Key Characteristics:

- Accessible via URLs (endpoints)
- Uses HTTP methods (GET, POST, PUT, DELETE)
- Platform and language independent
- Enables distributed systems to communicate

Example: When you check weather on your phone, the app calls a Web API like `api.weather.com/forecast?city=TelAviv`

Types of Web APIs

REST

- Most popular style
- Uses HTTP methods
- Stateless
- JSON/XML data format

SOAP

- XML-based protocol
- Strict standards
- Built-in security (WS-Security)
- Enterprise/legacy systems

Types of Web APIs

GraphQL

- Query language for APIs
- Client specifies data needed
- Single endpoint
- Developed by Facebook

gRPC

- High performance RPC
- Uses Protocol Buffers
- HTTP/2 based
- Microservices

What is REST?

REST - Representational State Transfer

REST is an **architectural style** (not a protocol) for designing networked applications. It was defined by Roy Fielding in his 2000 PhD dissertation.

Key Idea:

- Everything is a **resource** (user, book, order, etc.)
- Each resource has a unique **identifier** (URL)
- Resources are manipulated using standard **HTTP methods**
- Resources have **representations** (JSON, XML)

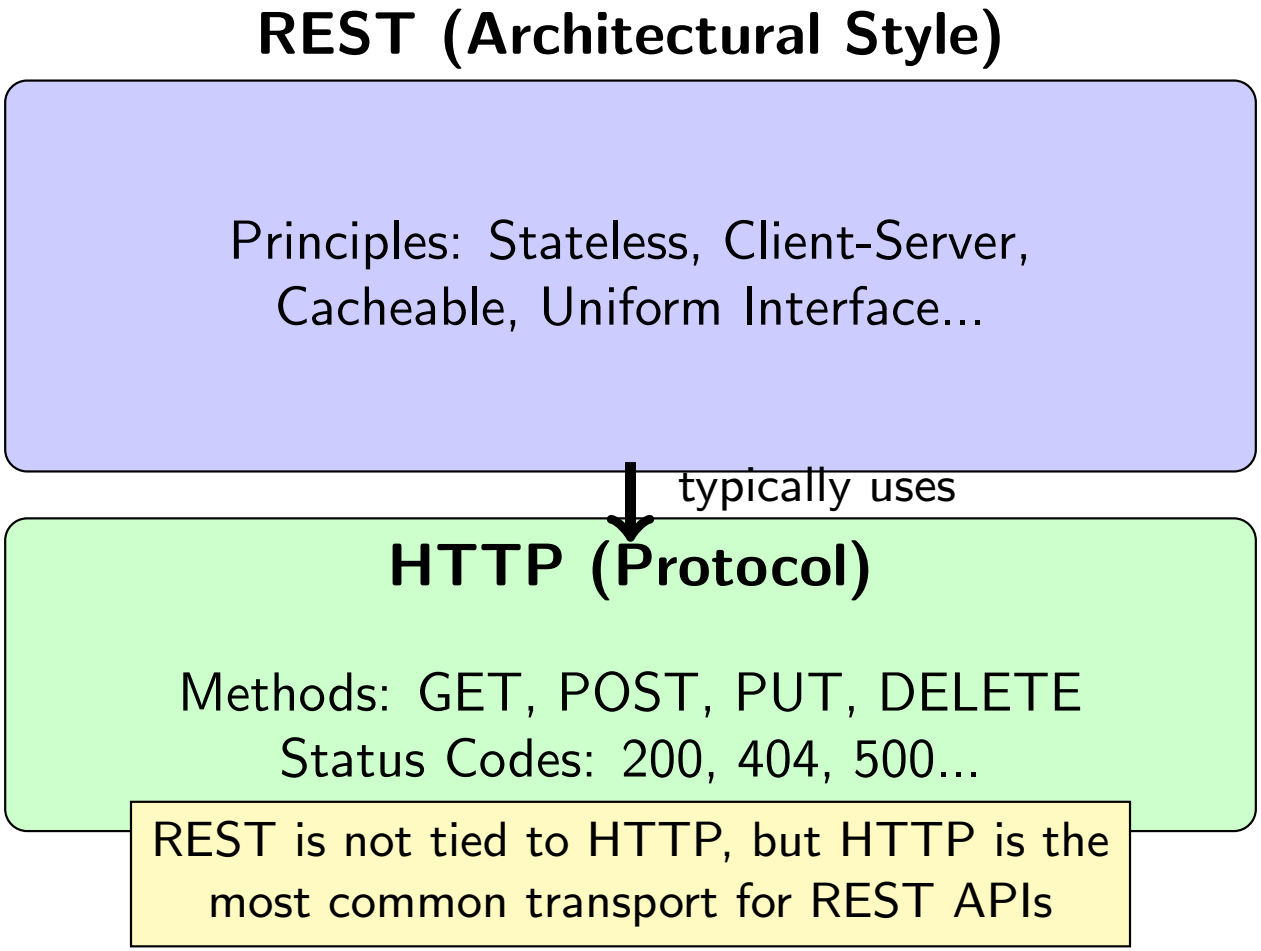
RESTful API = An API that follows REST principles

REST Principles (Constraints)

Six architectural constraints of REST:

1. **Client-Server:** Separation of concerns
   - Client handles UI, Server handles data storage
2. **Stateless:** Each request contains all needed information
   - Server doesn't store client session state
3. **Cacheable:** Responses can be cached
   - Improves performance and scalability
4. **Uniform Interface:** Consistent way to interact
   - Standard HTTP methods and status codes
5. **Layered System:** Client doesn't know if connected directly
   - Allows load balancers, proxies, gateways
6. **Code on Demand (optional):** Server can send executable code

REST and HTTP - The Relationship



RESTful URL Design

Resources are identified by URLs (endpoints):

URL Structure

```
https://api.example.com/v1/resources/identifier
```

Best Practices:

- Use **nouns**, not verbs: `/users` not `/getUsers`
- Use **plural** names: `/users` not `/user`
- Use **hierarchy** for relationships: `/users/123/orders`
- Use **query parameters** for filtering: `/users?role=admin`

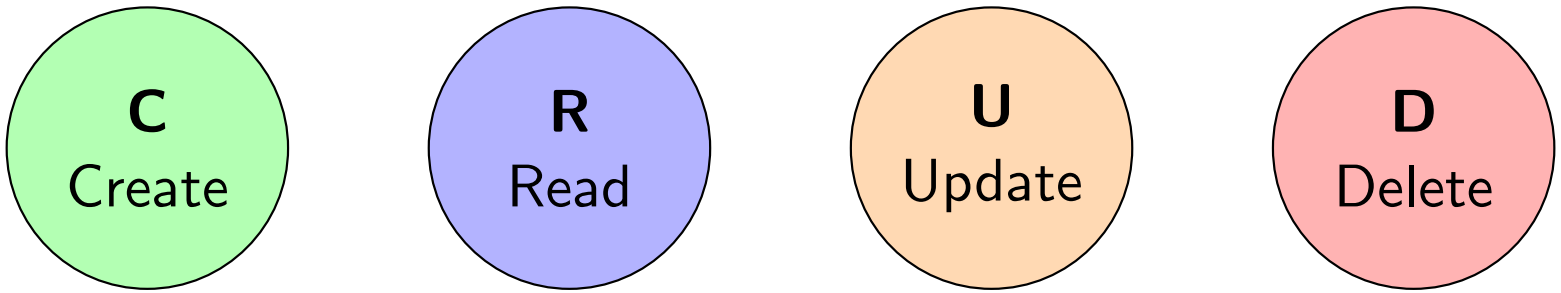
Examples:

- `GET /api/books` - Get all books
- `GET /api/books/978-0-123` - Get specific book
- `GET /api/users/42/orders` - Get orders for user 42

What is CRUD?

CRUD Definition

CRUD represents the four basic operations for persistent data storage:

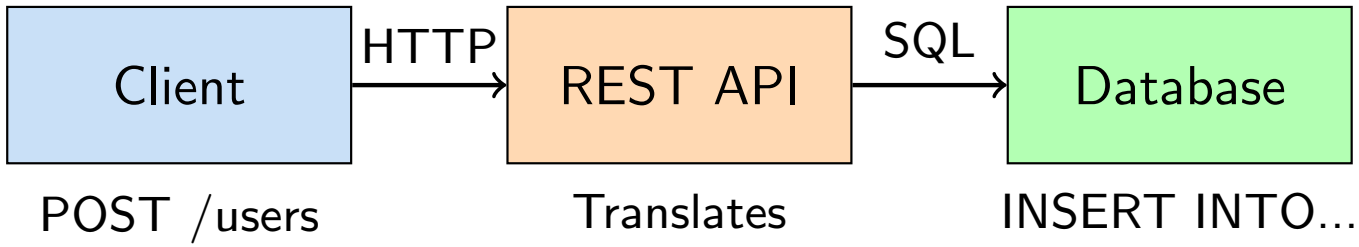


CRUD is Universal:

- Applies to databases, files, APIs, and more
- Foundation of most application functionality
- Maps directly to SQL and HTTP operations

CRUD Mapping Across Layers

Operation	SQL	HTTP Method	REST Example
Create	`INSERT`	`POST`	`POST /users`
Read	`SELECT`	`GET`	`GET /users/123`
Update	`UPDATE`	`PUT / PATCH`	`PUT /users/123`
Delete	`DELETE`	`DELETE`	`DELETE /users/123`



CREATE - POST Method

POST - Create a New Resource

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john@example.com"
}
```

Characteristics:

- Creates a **new** resource on the server
- Request body contains the new data
- Server generates ID and returns 201 `Created`
- **NOT idempotent**: Each call creates a new resource

CREATE - POST Response

Success Response (201 Created):

```
HTTP/1.1 201 Created
Location: /api/users/12345
Content-Type: application/json

{
  "id": "12345",
  "name": "John Doe",
  "email": "john@example.com",
  "created_at": "2025-12-01T10:30:00Z"
}
```

CREATE - POST Response

Success Response (201 Created): Key Points:

- `Location` header contains URL of new resource
- Response body includes server-generated fields (id, timestamps)
- Status 201 specifically means "resource created"

READ - GET Method

GET - Retrieve Resources

```
# Get all users
GET /api/users HTTP/1.1

# Get specific user
GET /api/users/12345 HTTP/1.1

# Get users with filtering
GET /api/users?role=admin&active=true HTTP/1.1
```

Characteristics:

- Retrieves resource(s) **without modifying** them
- **Idempotent**: Multiple identical requests = same result
- **Safe**: No side effects on the server
- Can return single resource or collection

UPDATE - PUT vs PATCH

PUT - Full Replacement

```
PUT /api/users/123 HTTP
    /1.1
Content-Type: application/
    json

{
  "name": "John Smith",
  "email": "john@new.com",
  "role": "admin"
}
```

- Replaces **entire** resource
- Must send ALL fields
- Idempotent

PATCH - Partial Update

```
PATCH /api/users/123 HTTP
    /1.1
Content-Type: application/
    json

{
  "email": "john@new.com"
}
```

- Updates **only specified** fields
- Send only what changed
- Usually idempotent

DELETE - DELETE Method

DELETE - Remove a Resource

```
DELETE /api/users/12345 HTTP/1.1
Host: api.example.com
```

Characteristics:

- Removes the specified resource
- **Idempotent**: Deleting twice = same result (resource gone)
- Returns `204 No Content` or `200 OK`
- No request body typically needed

DELETE - DELETE Method

Soft Delete vs Hard Delete:

- **Hard delete**: Actually removes from database
- **Soft delete**: Sets a "deleted" flag (data preserved)

CRUD Properties - Idempotency & Safety

Method	CRUD	Idempotent?	Safe?
GET	Read	✓Yes	✓Yes
POST	Create	× No	× No
PUT	Update	✓Yes	× No
PATCH	Update	✓Usually	× No
DELETE	Delete	✓Yes	× No

Definitions

- **Idempotent**: Multiple identical requests produce the same result as a single request
- **Safe**: The request does not modify server state

HTTP Status Codes in REST

2xx - Success

- `200 OK` - Success
- `201 Created` - Resource created
- `204 No Content` - Success, no body

3xx - Redirection

- `301 Moved Permanently`
- `304 Not Modified` - Use cache

HTTP Status Codes in REST

4xx - Client Errors

- `400 Bad Request` - Invalid input
- `401 Unauthorized` - Auth required
- `403 Forbidden` - No permission
- `404 Not Found` - Resource missing
- `409 Conflict` - State conflict

5xx - Server Errors

- `500 Internal Server Error`
- `502 Bad Gateway`
- `503 Service Unavailable`

What is a DTO?

Data Transfer Object (DTO)

A **DTO** is a design pattern - a simple object used to transfer data between systems or layers of an application.

Key Characteristics:

- Contains **only data**, no business logic
- Used to **encapsulate** data for transfer
- **Decouples** internal models from external representation
- Can be **serialized** to JSON, XML, or binary formats

Think of it as: A "shipping container" for data - standardized format for moving information between systems.

Why Use DTOs?

Benefits of using DTOs:

1. **Security - Control Exposed Data**
   - Hide sensitive fields (passwords, internal IDs)
   - Expose only what the client needs
2. **Performance - Reduce Payload Size**
   - Transfer only necessary fields
   - Avoid sending entire database objects
3. **Flexibility - API Versioning**
   - Change internal models without breaking API
   - Support multiple API versions simultaneously
4. **Clarity - Clear Contracts**
   - Define exactly what data is exchanged
   - Self-documenting data structures

DTO Example - Internal Model vs API Response

Internal User Model

```
{
  "id": 12345,
  "name": "John Doe",
  "email": "john@example.
      com",
  "password_hash": "a1b2c3
      ...",
  "ssn": "123-45-6789",
  "internal_flags": 42,
  "created_at": "2025-01-0
      1",
  "last_login_ip": "192.16
      8.1.1"
}
```

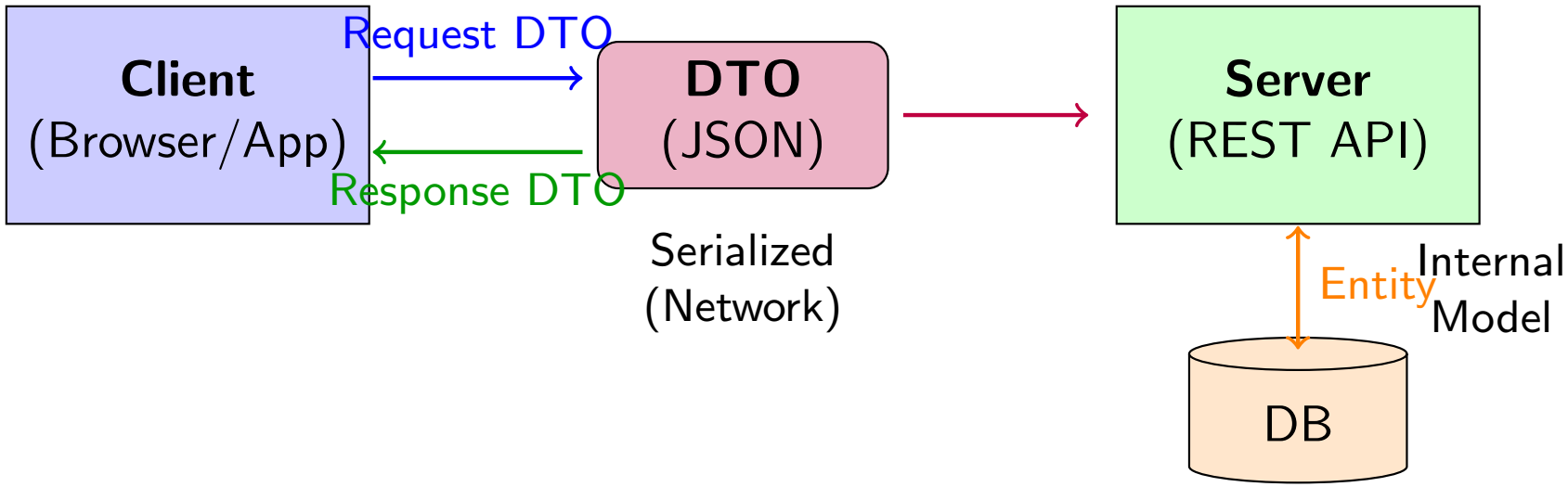
Contains sensitive data!

UserDTO (API Response)

```
{
  "id": 12345,
  "name": "John Doe",
  "email": "john@example.
      com",
  "member_since": "2025"
}
```

Only safe, relevant data exposed to clients.

DTOs in Client-Server Communication



Serialization: Converting DTO object → JSON/XML string for transmission
Deserialization: Converting JSON/XML string → DTO object for processing

Common DTO Serialization Formats

JSON

```
{
  "name": "John",
  "age": 30,
  "active": true
}
```

- Human-readable
- Native to JavaScript
- Most common for REST

XML

```
<user>
  <name>John</name>
  <age>30</age>
  <active>true</active>
</user>
```

- Verbose but structured
- Schema validation (XSD)
- Legacy/enterprise systems

Coming up next: Protocol Buffers - a more efficient binary format for DTOs

What are Protocol Buffers?

Protocol Buffers (protobuf)

Protocol Buffers is a language-neutral, platform-neutral mechanism for serializing structured data. Developed by Google.

Key Features:

- **Binary format** - not human-readable, but compact
- **Schema-defined** - data structure defined in `.proto` files
- **Code generation** - automatically generates classes for multiple languages
- **Backward/forward compatible** - can evolve schema safely

Connection to DTOs: Protobuf is a way to **define** and **serialize** DTOs with strong typing and efficiency.

Protobuf - Defining a DTO Schema

user.proto

```
syntax = "proto3";

message User {
  int32 id = 1;
  string name = 2;
  string email = 3;
  bool active = 4;
  repeated string roles = 5;  // list of strings
}

message UserList {
  repeated User users = 1;
}
```

Protobuf vs JSON - Comparison

JSON (45 bytes)

```
{
  "id": 123,
  "name": "John",
  "email": "j@ex.com",
  "active": true
}
```

Protobuf (20 bytes)

```
08 7B 12 04 4A 6F 68 6E
...
```

(Binary - not human readable)

Aspect	**JSON**	**Protobuf**
Size	Larger	**3-10x smaller**
Parse Speed	Slower	**5-100x faster**
Human Readable	✓Yes	× No
Schema Required	× No	✓Yes
Type Safety	Weak	**Strong**

Why Protobuf over JSON?

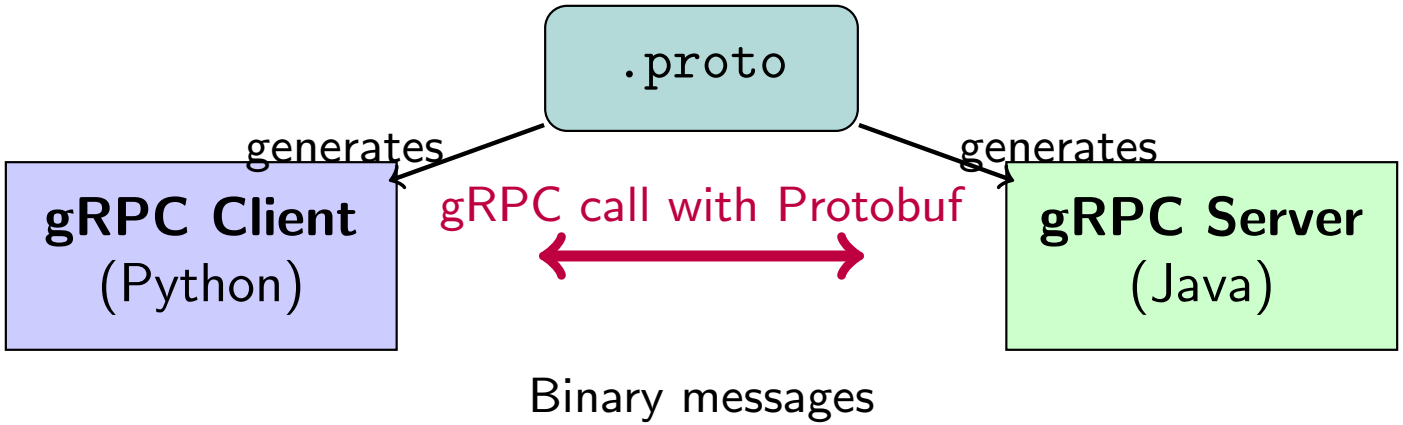
Advantages of Protocol Buffers

1. **Smaller Payload Size**
   - Binary encoding vs text
   - Field names not repeated in each message
2. **Faster Serialization/Deserialization**
   - No parsing of text format
   - Pre-compiled accessors
3. **Strong Typing**
   - Schema enforced at compile time
   - Catches errors early
4. **Schema Evolution**
   - Add new fields without breaking old clients
   - Remove fields safely with reserved tags

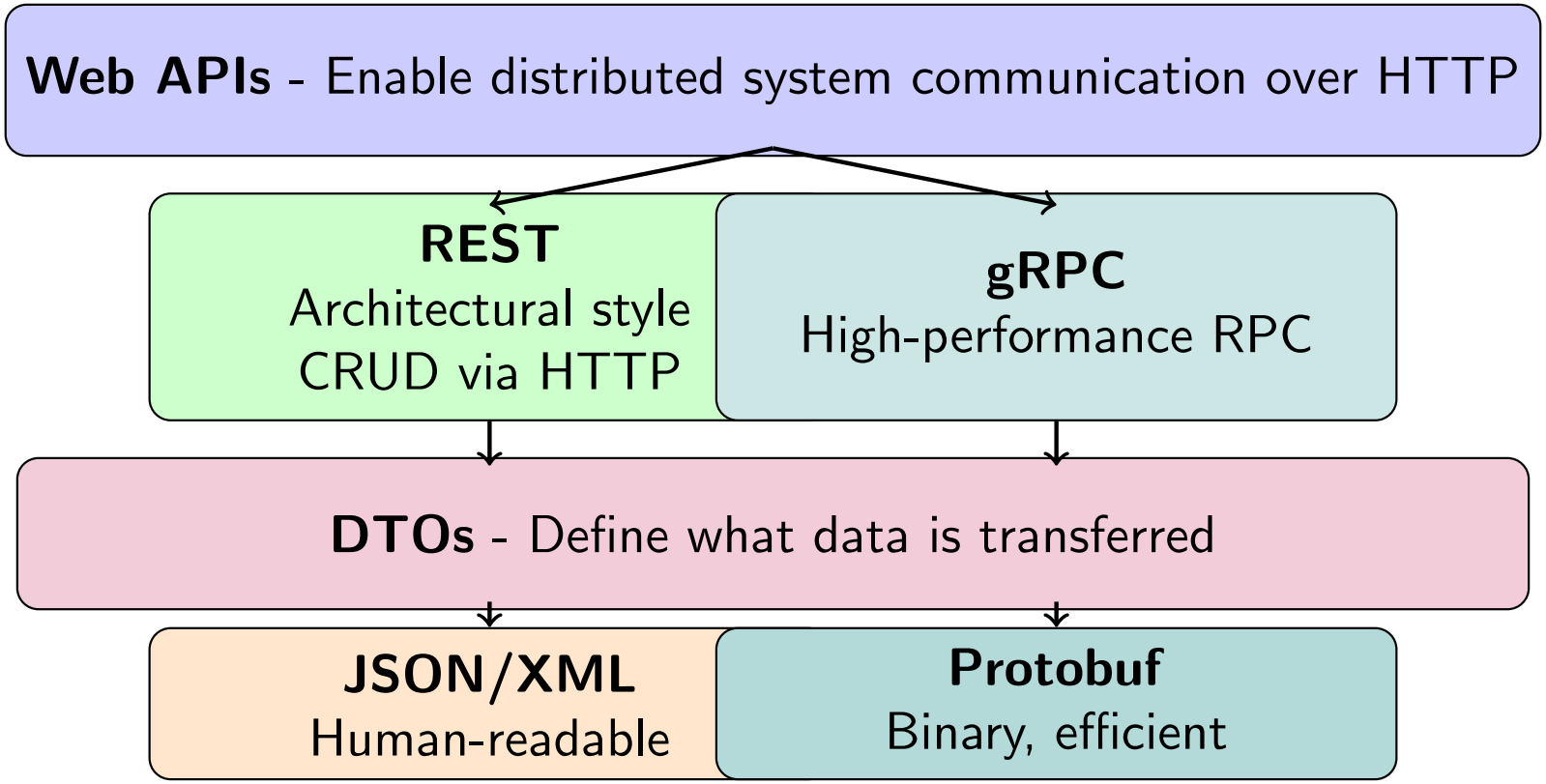
gRPC and Protocol Buffers

gRPC Overview

gRPC is a high-performance RPC (Remote Procedure Call) framework that uses Protocol Buffers by default.



How It All Connects



Design Exercise - Library Management System

Scenario: Design a REST API for a university library system.

System Requirements

The system needs to manage:

- **Books** - title, author, ISBN, availability status
- **Members** - name, email, membership ID
- **Loans** - which member borrowed which book, due date

Functionality:

1. Members can borrow and return books
2. Librarians can add, update, and remove books
3. Members can view their current loans
4. Search functionality for books

Task: Design the RESTful endpoints for this system.

Think About...

Before looking at the solution, consider:

1. What are the **resources**?
2. What **URLs** would you use for each resource?
3. Which **HTTP methods** for each operation?
4. What **status codes** for success and error cases?
5. How would you represent **relationships**?
   - A member's loans
   - Which book is on loan

Solution - Resources & Book Endpoints

Identified Resources:

Resource	Base URL	Description
Books	`/api/books`	Book inventory
Members	`/api/members`	Library members
Loans	`/api/loans`	Borrowing records

Book Endpoints:

Operation	Method + Endpoint	Description
List books	`GET /api/books`	Get all (paginated)
Search	`GET /api/books?title=...`	Filter books
Get one	`GET /api/books/{isbn}`	Book details
Add book	`POST /api/books`	Create record
Update	`PUT /api/books/{isbn}`	Update info
Remove	`DELETE /api/books/{isbn}`	Delete book

Solution - Member & Loan Endpoints

Member Endpoints:

Operation	Method + Endpoint	Description
List members	`GET /api/members`	Get all members
Get member	`GET /api/members/{id}`	Member details
Register	`POST /api/members`	New member
Update	`PATCH /api/members/{id}`	Update info
Member's loans	`GET /api/members/{id}/loans`	Active loans

Loan Endpoints:

Operation	Method + Endpoint	Description
Borrow book	`POST /api/loans`	Create loan
Return book	`PATCH /api/loans/{id}`	Mark returned
Loan details	`GET /api/loans/{id}`	View loan
Overdue	`GET /api/loans?status=overdue`	Filter overdue

Solution - Borrow Book Example

Request - Borrow a Book:

```
POST /api/loans HTTP/1.1
Content-Type: application/json

{
  "member_id": "M12345",
  "book_isbn": "978-0-13-468599-1",
  "due_date": "2025-12-15"
}
```

Solution - Borrow Book Example

Success Response (201 Created):

```
{
  "loan_id": "L98765",
  "member_id": "M12345",
  "book_isbn": "978-0-13-468599-1",
  "borrowed_date": "2025-12-01",
  "due_date": "2025-12-15",
  "status": "active"
}
```

Solution - Error Response Example

Error Response (409 Conflict - Book unavailable):

```
{
  "error": "BOOK_UNAVAILABLE",
  "message": "This book is currently on loan",
  "available_date": "2025-12-10"
}
```

Status Codes Used in This API:

Code	When Used
200 OK	Successful GET, PATCH
201 Created	Successful POST (new loan/book/member)
204 No Content	Successful DELETE
400 Bad Request	Invalid input data
404 Not Found	Book/Member doesn't exist
409 Conflict	Book already borrowed

Solution - Design Decisions Discussion

Key Design Decisions:

1. **Nested vs Flat URLs**
   - `GET /api/members/{id}/loans` - Shows clear relationship
   - `GET /api/loans?member_id={id}` - Also valid (flat)
   - Both approaches are RESTful
2. **Resource Identifiers**
   - Books: Use ISBN (natural key)
   - Members: Use membership ID
   - Loans: Auto-generated ID (no natural key)
3. **Why PATCH for Return?**
   - Only updating the `status` field
   - Alternative: `POST /api/loans/{id}/return` (action-based)
4. **Pagination**
   - `GET /api/books?page=2&limit=20`
   - Essential for large collections